

Architecture Sketch: Convergence Engine

```
flowchart LR
    U[Human challenge + constraints] --> P[Problem Framer]
    P --> R[Multi-level Retrieval]
    M[(Provenance Memory  
idea atoms + outcomes)] <--> R
    R --> D[Domain specialists  
parallel decomposition]
    D --> B[Analogue / Bridge agent  
structural isomorphisms]
    B --> G[Candidate population]
    G --> C[Critic + Red-team  
novelty · value · feasibility · evidence]
    C -->|retain / mutate| G
    C --> H[Human decision gate]
    H -->|reversible test approved| T[Prototype / simulation / tool runner]
    T --> O[Outcome + evidence ledger]
    O --> M
    H -->|reject / park| M
```

Operating pattern

Diverge: retrieve across abstraction levels; ask specialists to decompose mechanisms, not merely name analogies; pair domains deliberately far apart.

Converge: every candidate must state a mechanism, target beneficiary, falsifiable assumption, first reversible test, cost, and possible harm. Score it, then force a counterargument before it progresses.

Learn: persist the result of the test—not only the attractive idea. This turns the system from a novelty machine into a compounding innovation memory.

Agent contracts

Role	Produces	Cannot do alone
Problem framer	Goal, constraints, success/stop criteria	Invent or execute a solution
Specialist	Atom decomposition and domain evidence	Select final project
Analogue	Explicit source→target mapping	Claim a bridge is validated
Critic	Score and disconfirming evidence	Silence disagreement
Prototype agent	Reversible artifact/test result	Deploy or spend without approval
Human steward	Priority, permission, value judgment	Delegate accountability

Guardrails that preserve creativity

- Treat novelty and usefulness as separate signals; a high score in one does not compensate automatically for a near-zero score in the other.
- Maintain provenance: every claim points to a source atom, source document, experiment, or human decision.
- Use capability permissions: separate read, write, external communication, code execution, purchasing, and deployment.
- Set budget, time, data-access, and recursion limits per run.
- Require an explicit human gate for irreversible or external actions.